From MLPs to transformers: where universality changes

October 19, 2026

Abstract

The transition from an MLP to a transformer is often described by saying that transformers are more powerful or scale better. A more precise architectural distinction is that the sequence length moves from the shape of a flattened input weight matrix to the structure of the input itself. Parameter sharing and attention make this possible. Universal approximation theorems explain the resulting expressive capacity for each fixed sequence length; they do not establish a single set of universal weights for all lengths.

1 The starting point

Cybenko/Hornik (1989, 1991). Under the relevant activation assumptions, every continuous $f : K \subset \mathbb{R}^n \to \mathbb{R}^m$ on a compact set can be approximated arbitrarily closely by an MLP with one hidden layer. For hidden width w, the input weight matrix has wn entries.

Yun, Bhojanapalli, Rawat, Reddi, Kumar (2020). For each fixed sequence length T, transformers with trainable positional encodings can approximate any continuous $f : K \subset \mathbb{R}^{d\times T} \to \mathbb{R}^{d\times T}$ on a compact domain, in L^p for $1 \le p < \infty$. The numbers of heads, head size, and feed-forward hidden width can be fixed at $(h, m, r) = (2, 1, 4)$, independently of T and d. The construction can increase its depth. [1]

2 The architectural shift

Claim (the structural distinction)**.** *For a flattened-input MLP, the input weight matrix $W \in \mathbb{R}^{w\times(Td)}$ changes shape when T changes. For a transformer, the same attention and token-wise feed-forward projection matrices apply at each position. Their shapes do not depend on the sequence length T.*

In this sense, a dimension of the flattened input becomes a property of the input sequence:

$$\underbrace{Td}_{\text{MLP input-layer shape}} \longrightarrow \underbrace{T}_{\text{sequence structure}} . \tag{1}$$

This permits shared transformer blocks to process different sequence lengths. It is an architectural statement, distinct from the quantifiers in the approximation theorem:

$$\text{for every fixed } T,\ f,\ \epsilon > 0, \quad \text{there exists an approximating network } g_{T,f,\epsilon}.$$

The weights and depth may depend on T, f, and ϵ. Moreover, the trainable positional encoding in Yun is a $d \times T$ matrix and therefore has a parameter count that depends on T.

[1] The theorem uses a simplified transformer block without layer normalization. These small values of (h, m, r) do not mean that the total parameter count is independent of T or d.

3 The mechanism: contextual mapping and the division of roles

The constructive proof explains how the two types of layers divide the work.

Claim (division of roles)**.** *Self-attention layers compute contextual mappings. Token-wise feed-forward layers map the resulting contextual identifiers to target values.*

A *contextual mapping* associates each token with a scalar identifier that depends on the full input sequence. On the subset of discretized inputs used in the proof, these identifiers distinguish positions within an input and tokens across different inputs, subject to the relevant permutation symmetry. For example, occurrences of "I" in "I am happy" and "I am Bob" can receive different identifiers.

Yun constructs such a mapping on almost all points of a grid in $[0, 1]^{d\times T}$ using a sequence of self-attention layers. Each layer uses only pairwise interactions and shares its projections across positions, yet their composition encodes the full context. Feed-forward layers then map the distinguishable identifiers to the required outputs.

The contextual mapping explains how parameter sharing can coexist with universal approximation. The architectural shift describes the structure; the constructive proof supplies an expressive-capacity result.

Related empirical evidence. Yun also replaces attention layers with bilinear projections or depthwise separable convolutions. Replacing the first one or two attention layers in $\text{BERT}_{\text{BASE}}$ with convolutions can improve SQuAD and MNLI results, while replacing more layers hurts. These observations are consistent with cheap layers supplying some contextual information. They do not by themselves prove which mechanisms account for the remaining advantage of attention.

4 The accompanying mechanisms

1. **Parameter sharing across positions.** The same (W_Q, W_K, W_V) apply at each position. Their shapes remain fixed as the number of positions changes.

2. **Permutation equivariance before positional encoding.** Self-attention respects permutations of tokens. Positional encoding provides information about order. Without it, the universal approximation result is restricted to permutation-equivariant functions.

3. **Attention routing.** With full attention, each position can access every other position in one layer. A causal mask changes which interactions are available; the theorem discussed here concerns the paper's full-attention architecture.

5 What the theorem does not establish

Remark (computational cost)**.** *Dense attention conventionally forms $O(T^2)$ pairwise scores. Efficient implementations can reduce working memory, and sparse or linear attention changes the computation. Universality itself gives no guarantee of practical efficiency.*

Remark (the approximation resource: width and depth)**.** *In the classical shallow-MLP construction, width supplies increasing approximation capacity. Yun fixes $(h, m, r) = (2, 1, 4)$ and pays with depth. For example, the contextual-mapping construction uses $\delta^{-d} + 1$ attention layers for grid spacing δ. Fixed local widths therefore do not imply a small approximating network.*

Remark (universal approximation and Turing completeness)**.** *Yun concerns approximation of functions on a continuum. Results about Turing completeness concern computation and depend on assumptions about precision and available memory. They answer different questions.*

Remark (compact domains)**.** *The positional-encoding result applies on a compact domain. It does not guarantee uniform approximation over an unbounded input space.*

Remark (generalization to longer inputs)**.** *The existence of an approximator for each fixed T does not establish that a trained network will generalize from $T \leq T_{train}$ to much longer inputs. Length generalization is a separate question.*

Remark (positional information)**.** *The paper's extension beyond permutation-equivariant targets uses trainable positional encodings. The encoding is a substantive assumption in that result.*

6 Comparison by property

Property	Flattened-input MLP	Transformer
Approximation domain	Fixed $\mathbb{R}^n \to \mathbb{R}^m$	Fixed $\mathbb{R}^{d\times T} \to \mathbb{R}^{d\times T}$
Input projections	Depend on flattened input size	Shared across positions
Permutation equivariance	No built-in guarantee	Present before positional encoding
Approximation resource	Width in the shallow construction	Depth with fixed (h, m, r)
Long-range access	Through dense input connections	Through full attention
Variable input length	Requires adapting the input interface	Supported by shared blocks, subject to positional design
Dense interaction cost	Depends on layer widths	$O(T^2)$ attention scores

7 The same idea in other architectures

Sharing local parameters across a structural dimension of the input is a broader architectural idea:

- **CNNs:** convolution shares parameters across spatial positions and can operate on different spatial extents.
- **Graph neural networks:** shared message-passing rules operate on graphs with different numbers of vertices, with permutation equivariance over vertex labels.
- **Deep Sets:** shared element-wise maps and symmetric aggregation support permutation-invariant functions of sets.

Each architecture requires its own expressive-capacity theorem. Sharing parameters across a structural dimension does not automatically establish universal approximation or generalization across input sizes.

8 Conclusion

The architectural distinction is precise: transformer projections are shared across token positions, so their shapes need not grow with sequence length. Yun explains how attention and feed-forward layers can nevertheless approximate a rich class of functions for each fixed length. The fixed values $(h, m, r) = (2, 1, 4)$ describe local widths; the network's depth, weights, and positional encoding remain part of the approximation construction.

References

[1] Cybenko, G. *Approximation by superpositions of a sigmoidal function*. Mathematics of Control, Signals and Systems, 1989.

[2] Hornik, K. *Approximation capabilities of multilayer feedforward networks*. Neural Networks, 1991.

[3] Yun, C., Bhojanapalli, S., Rawat, A., Reddi, S., Kumar, S. *Are Transformers Universal Approximators of Sequence-to-Sequence Functions?*. ICLR 2020. arXiv:1912.10077.

[4] Zaheer, M. et al. *Deep Sets*. NeurIPS 2017.

[5] Pérez, J., Marinković, J., Barceló, P. *On the Turing Completeness of Modern Neural Network Architectures*. ICLR 2019.